Név:_____ Neptun:_____ Dátum:_____

Neumann
János
Egyetem

Programozás II.

1. ZH – Labor

A feladat egy könyvtár adatainak feldolgozása és kezelése egy program segítségével. A **konyvek.txt** fájl tartalmazza a könyvek adatait, minden mező szóközzel van elválasztva, és minden adatsor új sorba kerül. A fájl legfeljebb **100 adatsort** tartalmaz!

Készítsen egy *Konyv* osztályt, amely a példányok tárolására fog szolgálni. Az osztály minden adattagja legyen bárhonnan hozzáférhető. Az adattagok nevei a következők legyenek:

- ID – egész,
- Cim – karakterlánc,
- Szerzo – karakterlánc,
- Oldalszam – egész,
- Ar – egész,
- KiadasEve – egész.

Legyen az osztálynak paraméteres konstruktora, amely az adattagokat veszi át, és átadja a megfelelő változóknak. Legyen egy *ArKedvezmennyel* metódus, amely visszaadja a könyv árát a megadott százalékkal csökkentve. A kedvezmény mértékét paraméteresen vegye át, és csak egész típusú lehet a kedvezmény és a visszatérés is! Készítsen egy *OlvasasiIdo* metódust, amely visszaadja hány perc alatt olvasható el a könyv. A kiszámításhoz vegye alapul, hogy átlagosan 2 perc/oldal sebességgel olvas egy ember. Írja felül a ToString metódust, és az alábbi formátumban adja vissza a példány adatait. Ha nem tudja a ToStringgel megoldani, akkor a *Listaz* metódusban készítse el a kódrészletet a Konyvtar osztályban. (Pontlevonással jár) Minta: [Szerző]: [Név] ([ID]), kiadás éve: [Év], [Oldal] oldal, Ára: [Ár] Ft

Készítsen egy *Konyvtar* osztályt, amely a fentebb létrehozott osztálynak megfelelő *konyvek* tömbben tárolja az adatokat, amely bárhonnan elérhető. A tömb mérete a fájlnak megfelelő legyen. Készítsen egy *Beolvas* metódust, amely a tanult módon beolvassa a paraméterként a fájl nevét. Minden sorból készítsen egy példányt. Legyen egy *Listaz* metódus, amely kiírja az összes tárolt könyv adatát a fenti sablonnak megfelelően (ToString vagy itt megírt kód). Készítsen egy *Atlag* metódust, amely visszaadja a tárolt könyvek átlagos oldalszámát. Legyen egy *Legesleg()* metódus, amely visszaadja azt a *Konyv* példányt, amely a legolcsóbb a tömbben. Készítsen egy *Raforditott* metódust, amely kiírja annak a könyvnek az adatait, amely a legkevesebb idő alatt kiolvasható! Készítsen egy *Kereses* metódust, amely a paraméterül átvett érték szerint kilistázza az összes könyv adatát, amelynek a címe tartalmazza az átvett értéket vagy részletét.

Egészítse ki a main metódust. Kérje be a fájl elérését a felhasználótól, és tesztelje a Konyvtar osztály összes metódusát. Írja ki a metódusok meghívása előtt a metódus nevét vagy funkcióját!

A ZH megoldása során a projekt neve ZH legyen. Ha a feladat beadása után a program nem fordul le, vagy futási hiba van (szintaktikai hiba), akkor a dolgozat 0 pontot ér! Ahol szükséges, alkalmazzon specifikus hibakezelést. Minden fájl elejére a namespace után írja kommentbe be a nevét, neptun-kódját és a születési idejét. Amennyiben ezt elmulasztja, kettő pont levonással jár. Ha a projekt neve nem felel meg a fentebb leírtaknak, akkor további két pont levonással jár az elért pontszámból, illetve a javítási sor végére kerül a feladat.

Név:_____ Neptun:_____ Dátum:_____

Neumann
János
Egyetem

Programozás II.

1. ZH – Labor

A feladat egy könyvtár adatainak feldolgozása és kezelése egy program segítségével. A **konyvek.txt** fájl tartalmazza a könyvek adatait, minden mező szóközzel van elválasztva, és minden adatsor új sorba kerül. A fájl legfeljebb **100 adatsort** tartalmaz!

Készítsen egy *Konyv* osztályt, amely a példányok tárolására fog szolgálni. Az osztály minden adattagja legyen bárhonnan hozzáférhető. Az adattagok nevei a következők legyenek:

- ID – egész,
- Cim – karakterlánc,
- Szerzo – karakterlánc,
- Oldalszam – egész,
- Ar – egész,
- KiadasEve – egész.

Legyen az osztálynak paraméteres konstruktora, amely az adattagokat veszi át, és átadja a megfelelő változóknak. Legyen egy *ArKedvezmennyel* metódus, amely visszaadja a könyv árát a megadott százalékkal csökkentve. A kedvezmény mértékét paraméteresen vegye át, és csak egész típusú lehet a kedvezmény és a visszatérés is! Készítsen egy *OlvasasiIdo* metódust, amely visszaadja hány perc alatt olvasható el a könyv. A kiszámításhoz vegye alapul, hogy átlagosan 2 perc/oldal sebességgel olvas egy ember. Írja felül a ToString metódust, és az alábbi formátumban adja vissza a példány adatait. Ha nem tudja a ToStringgel megoldani, akkor a *Listaz* metódusban készítse el a kódrészletet a Konyvtar osztályban. (Pontlevonással jár) Minta: [Szerző]: [Név] ([ID]), kiadás éve: [Év], [Oldal] oldal, Ára: [Ár] Ft

Készítsen egy *Konyvtar* osztályt, amely a fentebb létrehozott osztálynak megfelelő *konyvek* tömbben tárolja az adatokat, amely bárhonnan elérhető. A tömb mérete a fájlnak megfelelő legyen. Készítsen egy *Beolvas* metódust, amely a tanult módon beolvassa a paraméterként a fájl nevét. Minden sorból készítsen egy példányt. Legyen egy *Listaz* metódus, amely kiírja az összes tárolt könyv adatát a fenti sablonnak megfelelően (ToString vagy itt megírt kód). Készítsen egy *Atlag* metódust, amely visszaadja a tárolt könyvek átlagos korát. Legyen egy *Legesleg()* metódus, amely visszaadja azt a *Konyv* példányt, amely a legdrágább a tömbben. Készítsen egy *Raforditott* metódust, amely kiírja annak a könyvnek az adatait, amely a legtöbb idő alatt kiolvasható! Készítsen egy *Kereses* metódust, amely a paraméterül átvett érték szerint kilistázza az összes könyv adatát, amelynek a szerzője tartalmazza az átvett értéket vagy részletét.

Egészítse ki a main metódust. Kérje be a fájl elérését a felhasználótól, és tesztelje a Konyvtar osztály összes metódusát. Írja ki a metódusok meghívása előtt a metódus nevét vagy funkcióját!

A ZH megoldása során a projekt neve ZH legyen. Ha a feladat beadása után a program nem fordul le, vagy futási hiba van (szintaktikai hiba), akkor a dolgozat 0 pontot ér! Ahol szükséges, alkalmazzon specifikus hibakezelést. Minden fájl elejére a namespace után írja kommentbe be a nevét, neptun-kódját és a születési idejét. Amennyiben ezt elmulasztja, kettő pont levonással jár. Ha a projekt neve nem felel meg a fentebb leírtaknak, akkor további két pont levonással jár az elért pontszámból, illetve a javítási sor végére kerül a feladat.